

demo

May 14, 2026

1 ANSER simulation in Python

A port of the MATLAB simulation with design considerations specific to python and future ML work.

1.0.1 Imports:

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from coil_dimension_calc import coil_dimension_calc
from build_field_generator import build_field_generator
from field_coil_calc import field_coil_calc
from sensor_objective_function import sensor_objective_function
from normal_vector import normal_vector
import consts
%matplotlib inline
```

1.1 Coil Geometry Plots

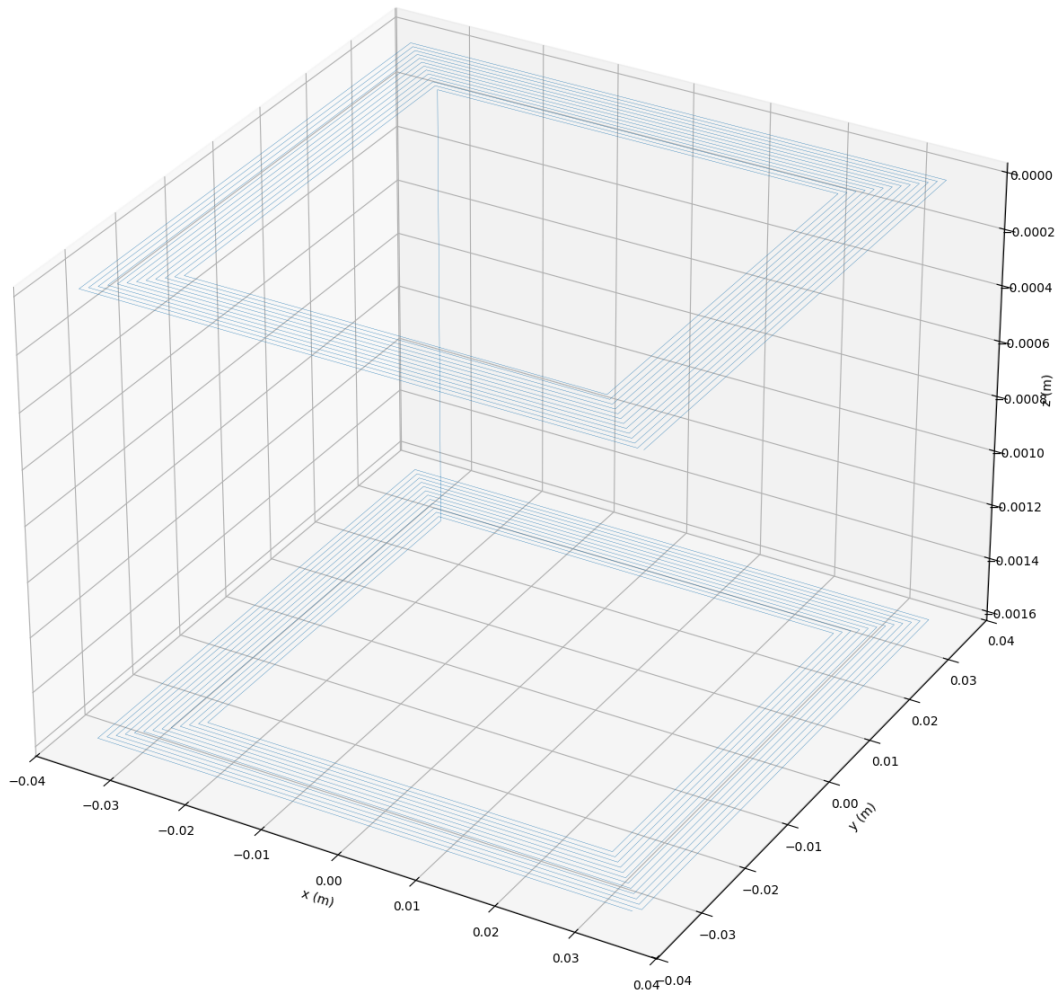
1.1.1 Single coil in local coordinate system

```
[2]: coil_local = coil_dimension_calc(consts.N_turns,consts.l,consts.w,consts.
    ↪s,consts.z_thick)

[3]: fig = plt.figure(figsize=(16,16))
ax = fig.add_subplot(111, projection='3d')
N = consts.N_turns

ax.plot3D(coil_local[:, 0], coil_local[:, 1], coil_local[:, 2], linewidth=0.25)

ax.set_xlabel('x (m)')
ax.set_ylabel('y (m)')
ax.set_zlabel('z (m)')
plt.show()
```



1.1.2 All 8 Coils in Global Coordinate System

```
[4]: coils_global = build_field_generator(consts.N_turns,consts.l,consts.w,consts.
    ↪s,consts.z_thick,consts.centres,consts.rotations)
```

```
[5]: print(coils_global.shape)
    coils = [f'Coil {i}' for i in range(coils_global.shape[0])]
```

```
(8, 102, 3)
```

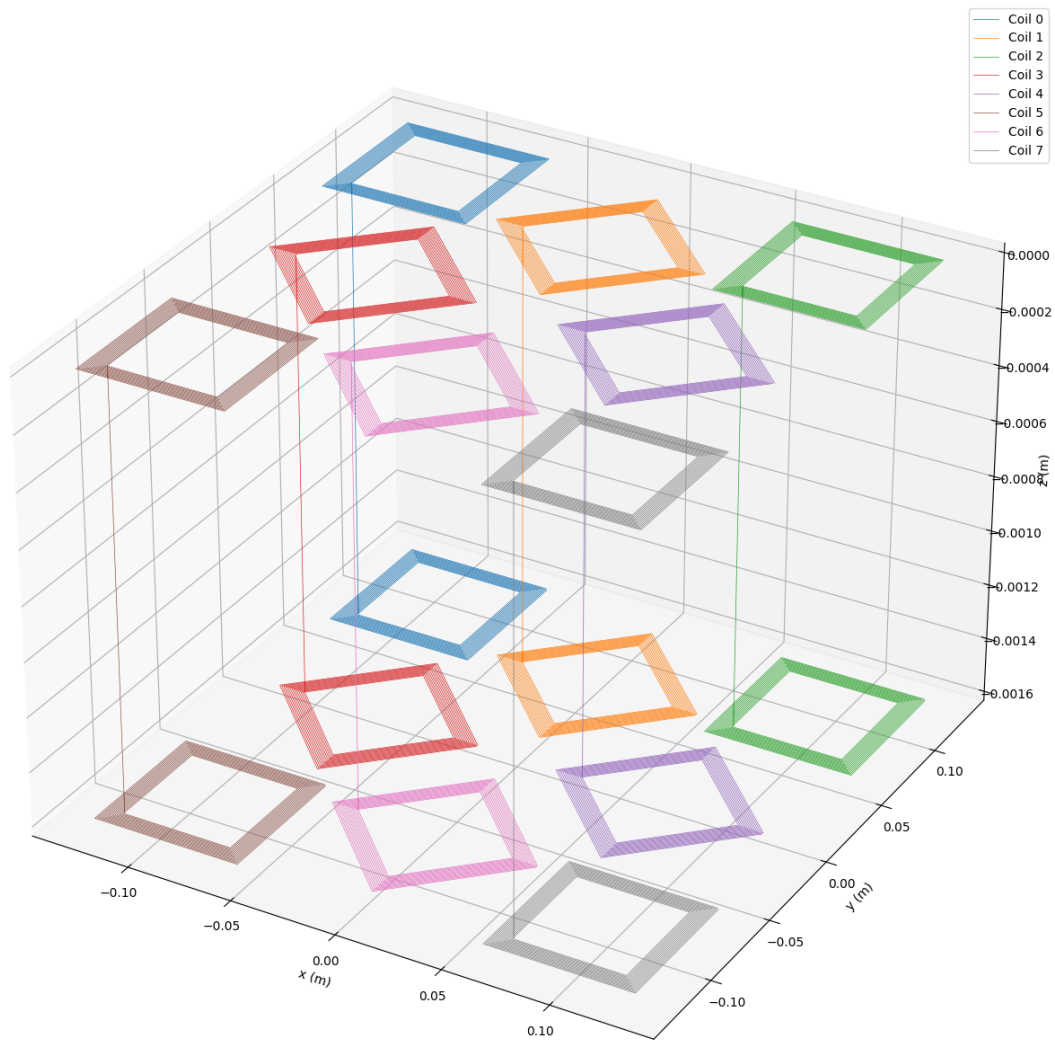
```
[6]: fig = plt.figure(figsize=(16,16))
    ax = fig.add_subplot(111, projection='3d')
    N = consts.N_turns
```

```

for i in range(coils_global.shape[0]):
    ax.plot3D(coils_global[i, :, 0], coils_global[i, :, 1], coils_global[i, :, 2], linewidth=0.5, label = coils[i])

ax.legend()
ax.set_xlabel('x (m)')
ax.set_ylabel('y (m)')
ax.set_zlabel('z (m)')
plt.show()

```



1.2 Compute H field at a test point

```
[7]: test_point = np.array([0.05,0.05,0.01])
```

```
[8]: test_field = field_coil_calc(consts.I, coils_global, test_point)
```

```
[9]: print(test_field)
```

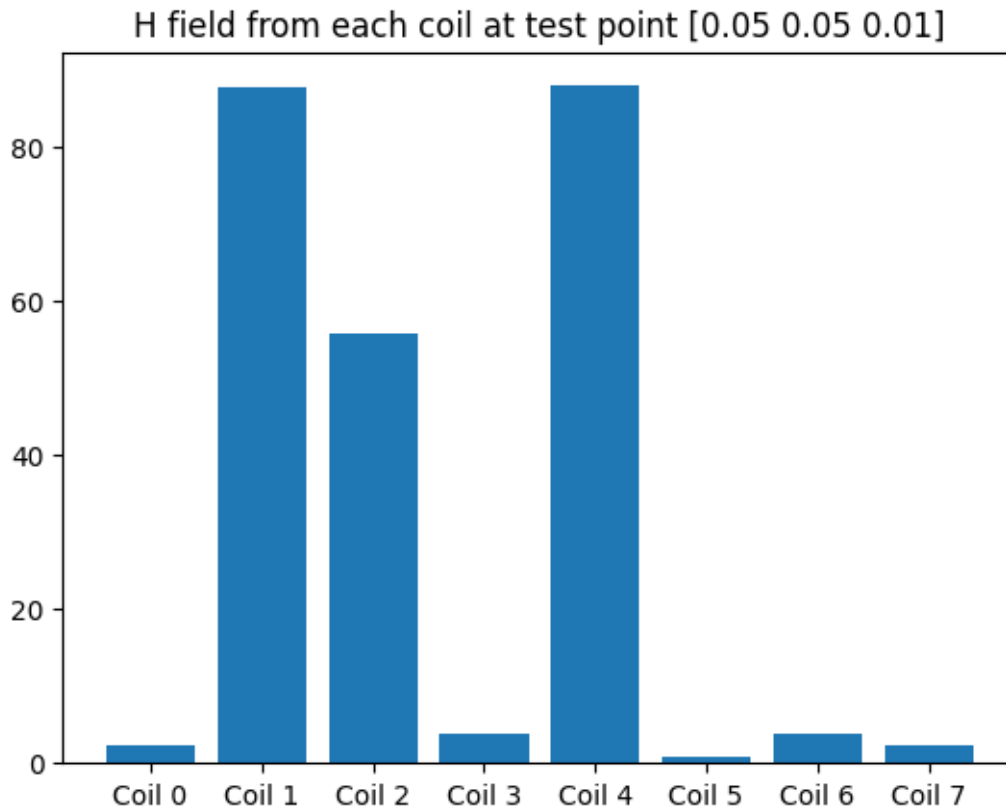
```
[[ -0.51089223   0.15941901   2.32420603]
 [-59.63289817  34.03541301  54.43563681]
 [ 28.30693569  28.16064046  38.94193955]
 [-0.93742983  -0.39132396   3.74383829]
 [ 33.94547093 -59.59124414  54.89780497]
 [-0.11000052  -0.10487321   0.92177669]
 [-0.40618861  -0.92828942   3.74876638]
 [  0.14620048  -0.50674701   2.3303384  ]]
```

```
[10]: test_field_norm = np.linalg.norm(test_field, axis = -1)
```

```
[11]: print(test_field_norm)
```

```
[ 2.38502809  87.62265938  55.77435733  3.87920537  87.84748354  0.93422202
  3.88329239  2.38927691]
```

```
[12]: plt.bar(coils,test_field_norm)
plt.title(f'H field from each coil at test point {test_point}')
plt.show()
```



1.3 Solve for a test point using least squares.

1.3.1 Sweep over noise

```
[18]: %%time
from scipy.optimize import least_squares

test_P0 = np.array([0.1,0.1,0.1,np.pi/3,np.pi/2])

H = field_coil_calc(consts.I,coils_global,test_P0[:3])
n = normal_vector(test_P0)

flux = consts.calibration * np.sum(H*n, axis = -1)

snrs = np.arange(10, 65, 5)
errors = []

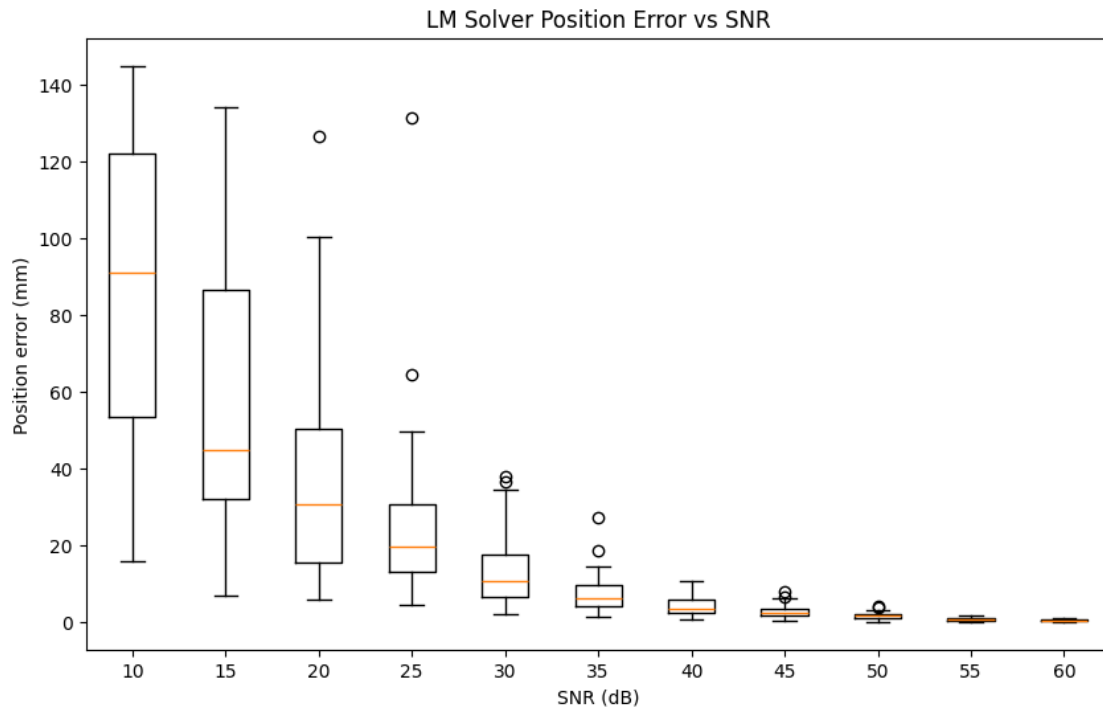
for snr in snrs:
    errs = []
    for _ in range(50):
```

```

    flux_noisy = flux + np.sqrt(np.mean(flux**2) / 10**(snr/10)) * np.
    random.randn(8)
    result = least_squares(
        sensor_objective_function,
        x0=[0, 0, 0.2, 0, 0],
        bounds=(-0.5, -0.5, 0, -np.pi, -3*np.pi],
                [0.5, 0.5, 0.5, np.pi, 3*np.pi]),
        args=(flux_noisy, coils_global, calibration),
    )
    errs.append(np.linalg.norm(result.x[:3] - p) * 1000)
errors.append(errs)

plt.figure(figsize=(10, 6))
plt.boxplot(errors, tick_labels=snrs)
plt.xlabel('SNR (dB)')
plt.ylabel('Position error (mm)')
plt.title('LM Solver Position Error vs SNR')
plt.show()

```



CPU times: user 24.7 s, sys: 3.98 ms, total: 24.7 s
Wall time: 24.9 s

Here we did a sweep over 11 difference noise values ranging from 10 to 60. For each we do 50 simulations and count the mean error per simulation. Least squares struggles with noisy data, and takes approximately 0.05 seconds for each solve on a reasonably good computer.

[]: